TP Unidad 7 — NoSQL y nuevas formas de almacenamiento

Proyecto: Food Store · **Asignatura:** Bases de Datos II (UTN) · **Motores:** PostgreSQL 18 (`food_store_tp3`) y MongoDB (colección documental).
Subconjunto: 5 pedidos existentes con sus detalles (IDs 1–5). **Código adjunto:** `migracion_pedidos.js`, `consultas_mongodb.js`, `consultas_postgresql.sql` (+ `ejecutar_mongodb.py`).

> **Ejecución real:** migración y consultas (a)–(d) ejecutadas **realmente** contra MongoDB **9.0.4** en `127.0.0.1:27017` (servicio activo) con pymongo (`ejecutar_mongodb.py`, equivalente a `insertMany/find`; los `.js` también son ejecutables con `mongosh --file`). Evidencia: `evidencia_mongodb.txt`. Verificación relacional ejecutada en PostgreSQL (`evidencia_postgresql.txt`): coincide en cantidad e IDs.

Parte 1 — Modelo documental y migración

Consigna 1. Tablas/claves en PostgreSQL (versión real) y esquema documental

- `pedido(id PK, fecha timestamptz, forma_pago enum, cliente_id FK→cliente.id, created_at, eliminado bool)`.
- `detalle_pedido(id PK, pedido_id FK→pedido.id, producto_id FK→producto.id, cantidad int, precio_unitario numeric, subtotal numeric, eliminado, UNIQUE(pedido_id,producto_id))`.
- `cliente(id PK, nombre, apellido, email UNIQUE, telefono, created_at)` y `producto(id PK, nombre,...)` (se toma el nombre).

Relaciones: pedido N:1 cliente; pedido 1:N detalle_pedido; detalle N:1 producto. **Colección:** `food_store_nosql.pedidos` (1 documento = 1 pedido completo).

Campo	Forma	Origen / nota
`_id`, `id_origen`	entero	`pedido.id` preservado (trazabilidad)
`fecha`, `forma_pago`, `estado`	ISODate, string, string	`estado` derivado de `eliminado` (ACTIVO/ANULADO)
`total`	Decimal128	Suma de subtotales históricos (almacenado para consulta)
`cliente`	**objeto embebido**	`{id_origen,nombre,apellido,email}` (snapshot)
`lineas`	**arreglo** de objetos	`{producto_id,producto,cantidad,precio_unitario,subtotal}`

Consigna 2. Migración (≥ 5 pedidos)

`migracion_pedidos.js`: `use('food_store_nosql')`, `db.pedidos.drop()`, `db.pedidos.insertMany([...])` con 5 documentos (ids 1–5) que incluyen fecha, estado, cliente, productos, cantidades y **precios históricos** (`precio_unitario`, `subtotal`, `total`), conservando identificadores de origen. Datos extraídos de PostgreSQL.

Consigna 3. Reglas de integridad y decisión de embebido

1. **Precios/cantidades históricos inmutables:** `precio_unitario`/`subtotal` se copian desde `detalle_pedido` (no se recalculan con el precio actual de `producto`), preservando lo facturado (regla R4). `total` coherente con esos subtotales.
2. **Trazabilidad y pertenencia pedido–líneas:** se preservan `id_origen`, `cliente.id_origen` y `producto_id`; las líneas quedan dentro del documento del pedido (equivale a la FK `detalle_pedido.pedido_id`), evitando líneas huérfanas.

Embebido vs colecciones separadas: se embeben `cliente` y `lineas` porque el acceso frecuente es el *pedido completo* en una sola lectura (sin `$lookup`/JOIN), patrón recomendado para agregados contenidos (MongoDB, *Embedded Data Models*). No se embebe el catálogo maestro completo: solo el snapshot necesario, manteniendo coherencia histórica.

Parte 2 — Consultas y verificación

Consigna 4. Consultas MongoDB (`consultas_mongodb.js`)

- **(a)** `find({}, {id_origen:1, estado:1})` → ids 1–5, `ACTIVO`.
- **(b)** `find({estado:"ACTIVO"})` → estado `ACTIVO`, **cantidad 5**, IDs `1,2,3,4,5`.
- **(c)** `find({"lineas.producto":"Producto 16"})` → **cantidad 1**, ID `2`.
- **(d)** `find({total: {$gt: NumberDecimal("30000")}})`. **Umbral 30000** (totales 27724,48; 45193,22; 49020,34; 21370,25; 35496,55; promedio ≈35760,93): separa tickets bajos (1,4) de altos (2,3,5). → **cantidad 3**, IDs `2,3,5`.

Consigna 5. Equivalentes PostgreSQL y comparación (ejecución real)

Consultas (b),(c) en `consultas_postgresql.sql` (acotadas a ids 1–5). Ejecución real (`evidencia_postgresql.txt`):

```
(b) ACTIVO  -> cantidad 5, ids {1,2,3,4,5}
(c) "Producto 16" -> cantidad 1, ids {2}
```

Consulta	Cantidad PostgreSQL	Cantidad MongoDB	Coinciden IDs
Estado seleccionado: ACTIVO	5	5	**Sí** — {1,2,3,4,5}
Producto seleccionado: Producto 16	1	1	**Sí** — {2}

Diferencias: ninguna. Coinciden porque la migración preservó `id_origen` y valores históricos/estado, y los filtros son equivalentes (`eliminado=false` ↔ `estado:"ACTIVO"`; JOIN por nombre ↔ `lineas.producto` embebido). No se requirió corrección.

Parte 3 — Evaluación de la tecnología e IA

Consigna 6. Consulta a IA y respuesta (síntesis)

Consulta: “En Food Store (PostgreSQL: pedido/detalle_pedido/cliente/producto), con acceso frecuente a pedidos completos y necesidad de coherencia de cantidades y precios históricos, compará PostgreSQL y MongoDB para almacenar y consultar pedidos.”

Afirmación 1 (IA): “MongoDB obtiene un pedido completo (cliente y líneas) en una sola consulta al embeber, evitando los JOIN de PostgreSQL, lo que suele mejorar la lectura de agregados.”

Afirmación 2 (IA): “PostgreSQL garantiza integridad referencial y restricciones (FK, CHECK, UNIQUE) nativas; en MongoDB esa integridad recae en la aplicación/validación de esquema, por lo que para cantidades y precios coherentes el relacional es más seguro por defecto.”

Consigna 7. Contraste y conclusión

Contraste A1: confirmada. MongoDB documenta *embedded data models* para recuperar datos relacionados en una operación y evitar `$lookup` al consultar el agregado (Manual MongoDB, *Embedded Data Models*). En el laboratorio, (a)–(d) resuelven pedido+cliente+líneas con un `find`; el equivalente (c) en PostgreSQL requirió JOIN pedido–detalle–producto. Matiz: depende del patrón de acceso.

Contraste A2: confirmada. El esquema real usa FK, `UNIQUE(pedido_id,producto_id)` y CHECK de cantidades/precios (`schema.sql`), impuestos por el motor. MongoDB ofrece `$jsonSchema` pero no FK nativas entre documentos; la coherencia (p. ej. `total` vs suma de `subtotal`) recae en la escritura/migración (verificada contra la fuente). La garantía estructural por defecto es mayor en PostgreSQL.

Conclusión (≤200 palabras, 2 criterios). Conviene **mantener PostgreSQL como fuente de verdad** y usar **MongoDB solo como vista documental complementaria** de lectura (no reemplazo). *Criterio 1 — consistencia/integridad:* pedidos manejan importes y relaciones; PostgreSQL impone FK, CHECK, UNIQUE y transacciones ACID nativas, reduciendo incoherencias que en MongoDB dependen de la aplicación/validación. *Criterio 2 — patrón de acceso:* el pedido completo se lee en MongoDB con un documento embebido (una lectura, sin JOIN), útil para consulta de agregados, como mostró el laboratorio con resultados idénticos a PostgreSQL. La ganancia es de modelo de lectura, no de corrección: la coherencia histórica se preservó por respetar la fuente. Opción equilibrada: transaccional/normativa en

PostgreSQL y proyección documental en MongoDB para agregados, sincronizada desde el origen relacional.

Identificadores de origen preservados; sin datos personales reales (entorno de desarrollo). Referencias: MongoDB Manual — *Embedded Data Models*, *db.collection.find()*.